Variables de entorno y primera petición HTTP autenticada (Bloque 4) / Tu primer disparo a una API, desde Postman o Thunder Client

✓ **Hecho:** Ver

📘 **UNIDAD U1.2** · Setup técnico: terminal, Git, entorno Python y cliente HTTP

Tu primer disparo a una API, desde Postman o Thunder Client

Con tu archivo "**.env**" listo y tu credencial protegida, falta comprobar que esa credencial realmente funciona antes de escribir una sola línea de código que dependa de ella.

Esta lección explica cómo se hace esa comprobación, usando un cliente **HTTP** como **Postman** o **Thunder Client**, para que en la siguiente lección de este mismo bloque la ejecutes tú mismo como cierre de la unidad.

☝️ **Una aclaración antes de seguir**: para esta prueba no necesitas tener activado el entorno virtual que armaste en el **Bloque 3**. Estás usando un cliente con interfaz gráfica (**Postman** o **Thunder Client**), no ejecutando código Python, así que tu entorno virtual todavía no entra en juego. Vuelve a ser relevante recién en la **Unidad 1.3**, cuando lo actives para correr el módulo **Python** que consuma esta misma API.

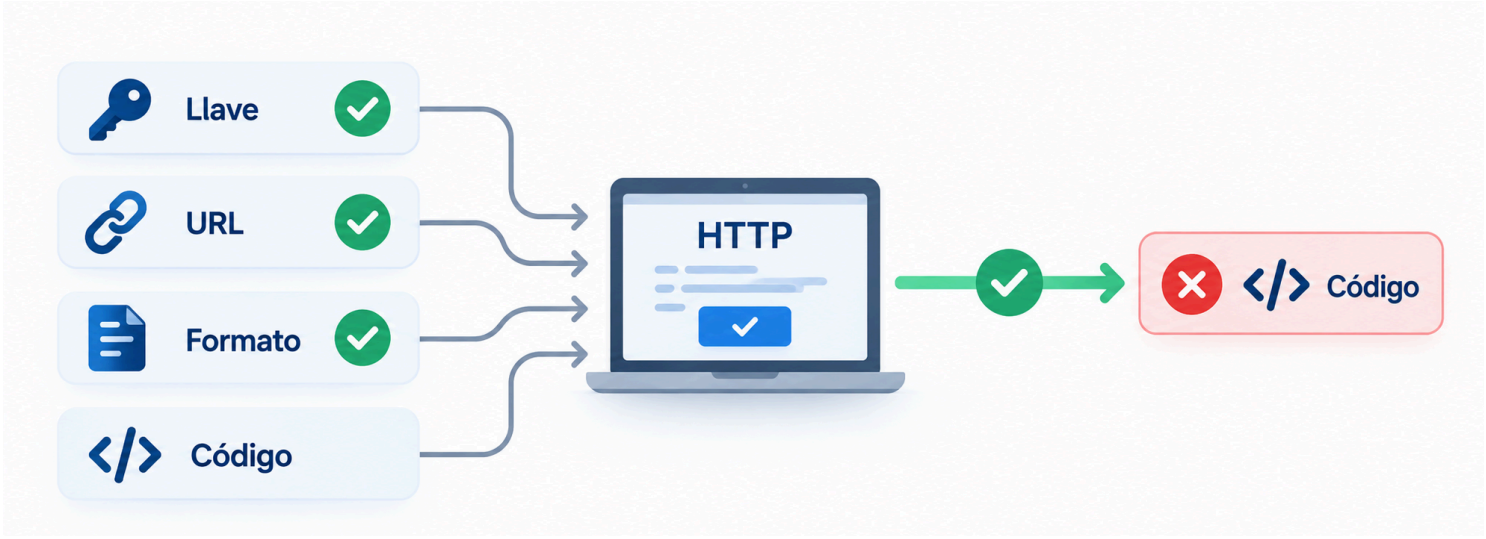
🛠️ Qué son estas herramientas, y por qué conviene probar antes de programar

Postman y Thunder Client son clientes HTTP: programas con una interfaz gráfica que arman y envían peticiones a una API sin que tengas que escribir código para hacerlo.

La razón para probar primero con uno de estos clientes, en vez de ir directo a escribir el código que consume la **API**, es aislar el problema. Si tu primera petición falla mientras programas, puede ser por muchas razones a la vez: la llave, la **URL**, el formato de la petición, o un error en tu propio código.



Probar antes con un cliente HTTP elimina la variable del código: si la petición funciona ahí, sabes con certeza que tu credencial y tu configuración están bien, y cualquier problema posterior al programarlo va a estar en tu código, no en tu setup.



⚖️ Postman o Thunder Client: en qué se diferencian

Los dos cumplen exactamente el mismo propósito en esta lección, pero no son lo mismo. Antes de elegir, vale la pena conocer las diferencias reales entre ambos:



Dónde viven

⚡ Thunder Client

Es una extensión que vive directamente dentro de **VS Code**, como una pestaña más del editor: nunca sales de tu flujo de trabajo.

🚀 Postman

Es una aplicación aparte (de escritorio o web), separada del editor, a la que tienes que cambiarte.



Si te piden crear una cuenta

⚡ Thunder Client

Se puede usar sin registrarte para lo básico que necesitas en esta unidad, con tus datos guardados localmente en tu computador.

🚀 Postman

Sí te pide crear una cuenta gratuita e iniciar sesión para usar la aplicación de escritorio.



Complejidad y funciones

🚀 Postman

Es más completo: colecciones avanzadas, pruebas automatizadas, servidores simulados, espacios de equipo compartidos. Es una herramienta pensada también para proyectos grandes y equipos.

⚡ Thunder Client

Es más liviano y directo, con lo esencial para armar y probar una petición, sin una curva de aprendizaje larga.



Para esta unidad

⚡ Thunder Client

Si prefieres evitar crear una cuenta adicional y quedarte dentro de **VS Code**, Thunder Client es la opción con menos fricción.

🚀 Postman

Si ya usas **Postman**, o sabes que lo vas a necesitar más adelante en otros proyectos, no hay ninguna razón para no empezar con él aquí.

Ambos producen exactamente el mismo resultado para lo que esta lección te pide.

🧩 La anatomía de una petición HTTP

Toda petición que armes en uno de estos clientes tiene las mismas partes, sin importar qué API sea:



El método



La URL del



Los



El cuerpo (body)

Indica qué tipo de acción se pide (la más común para consultar o pedir algo a un modelo es la que envía datos y espera una respuesta generada).

endpoint

La dirección exacta del servicio de tu proveedor de modelo al que le estás hablando, tal como aparece en su documentación.

encabezados (headers)

Información adicional que acompaña la petición sin ir en el contenido principal; aquí es donde normalmente viaja tu credencial de autenticación.

El contenido real de tu pedido cuando corresponde, casi siempre en formato **JSON**: por ejemplo, el texto que le quieres enviar al modelo y los parámetros de la petición.

💻 Dónde se ingresa cada cosa en la interfaz

Aunque la interfaz exacta varia ligeramente entre herramientas, el procedimiento paso a paso en ambas sigue el mismo flujo claro:

1

Seleccionar el método y la URL

En la barra superior, cambia el desplegable del método a **POST** y pega la URL completa del endpoint provisto por tu plataforma.

2

Configurar encabezados (Headers)

Haz clic en la pestaña **"Headers"**. En la columna **"Key"**, escribe el nombre del encabezado (por ejemplo, Content-Type o Authorization). En la columna **"Value"**, ingresa su valor respectivo (por ejemplo, application/json o Bearer TU_LLAVE_AQUI).

3

Ingresar el cuerpo (Body)

Pasa a la pestaña **"Body"**. Si usas Postman, selecciona la opción **"raw"** y asegúrate de cambiar el tipo de contenido desplegable a **"JSON"**. En Thunder Client, selecciona la pestaña **"JSON"** directamente. Pega en el editor el bloque de código con el modelo y el mensaje.

Un ejemplo genérico ayuda a visualizarlo, aunque el formato exacto (los nombres de los campos, cómo se llama el encabezado de autenticación) cambia según el proveedor que elijas: revisa siempre la documentación específica del tuyo.

Un encabezado típico se ve así:

```
Content-Type: application/json
 Authorization: Bearer TU_LLAVE_AQUI
```

Y un cuerpo típico, así:

```
{
   "model": "nombre-del-modelo",
   "messages": [
     { "role": "user", "content": "Hola, ¿puedes confirmar que esta conexión funciona?" }
   ]
 }
```

⚠️ "**TU_LLAVE_AQUI**" y "**nombre-del-modelo**" son marcadores de ejemplo, no los escribas literalmente: reemplázalos por tu credencial real (copiada desde tu "**.env**") y por el nombre de modelo que indique la documentación de tu proveedor.

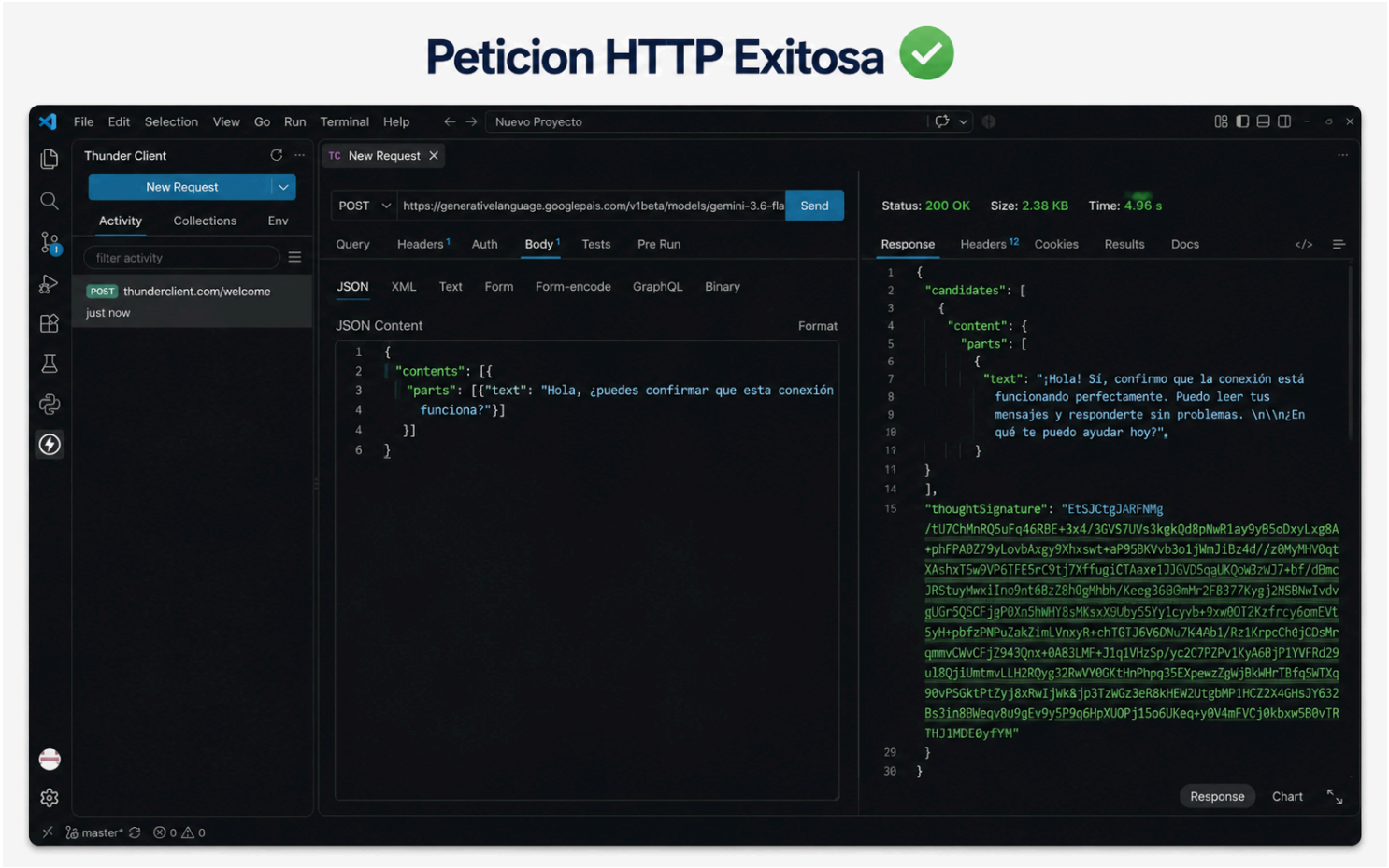
Dónde va tu credencial en todo esto

Tu llave de **API** casi nunca va en la **URL** ni en el cuerpo de la petición: va en los encabezados, siguiendo el formato que indique la documentación de tu **proveedor** de modelo (algunos usan un encabezado de autorización con un formato específico, otros usan un encabezado con nombre propio para la llave).

```
# OpenAI, Groq, Mistral AI, Anthropic (Claude), Hugging Face
Authorization: Bearer TU_LLAVE_AQUI

# Google Gemini API (la excepción: no usa "Authorization")
x-goog-api-key: TU_LLAVE_AQUI
```

Aquí es donde se conecta todo lo que trabajaste en la lección anterior: en vez de escribir la llave real dentro del cliente **HTTP**, la copias desde tu archivo ".env", exactamente el mismo valor que tu código va a leer más adelante desde esa variable de entorno.



⚠ Los errores más comunes en esta primera prueba, y qué dicen realmente

Casi todos los tropiezos de esta primera prueba caen en un puñado de causas conocidas, y el código de estado de la respuesta suele señalar directamente cuál de ellas es:

Código 401 · Autenticación

Casi siempre significa que la credencial no llegó bien: copiada a medias, con un espacio

Código 404 · Recurso no encontrado

Suele indicar que la **URL** del endpoint tiene un error de tipeo, o que corresponde a una

Código 429 · Demasiadas peticiones

Significa que tu credencial es válida, pero excediste el límite de uso permitido en un periodo

de más, o puesta en el encabezado equivocado.

Vuelve a copiarla directamente desde tu archivo **".env"**, sin escribirla a mano.

versión distinta de la **API**

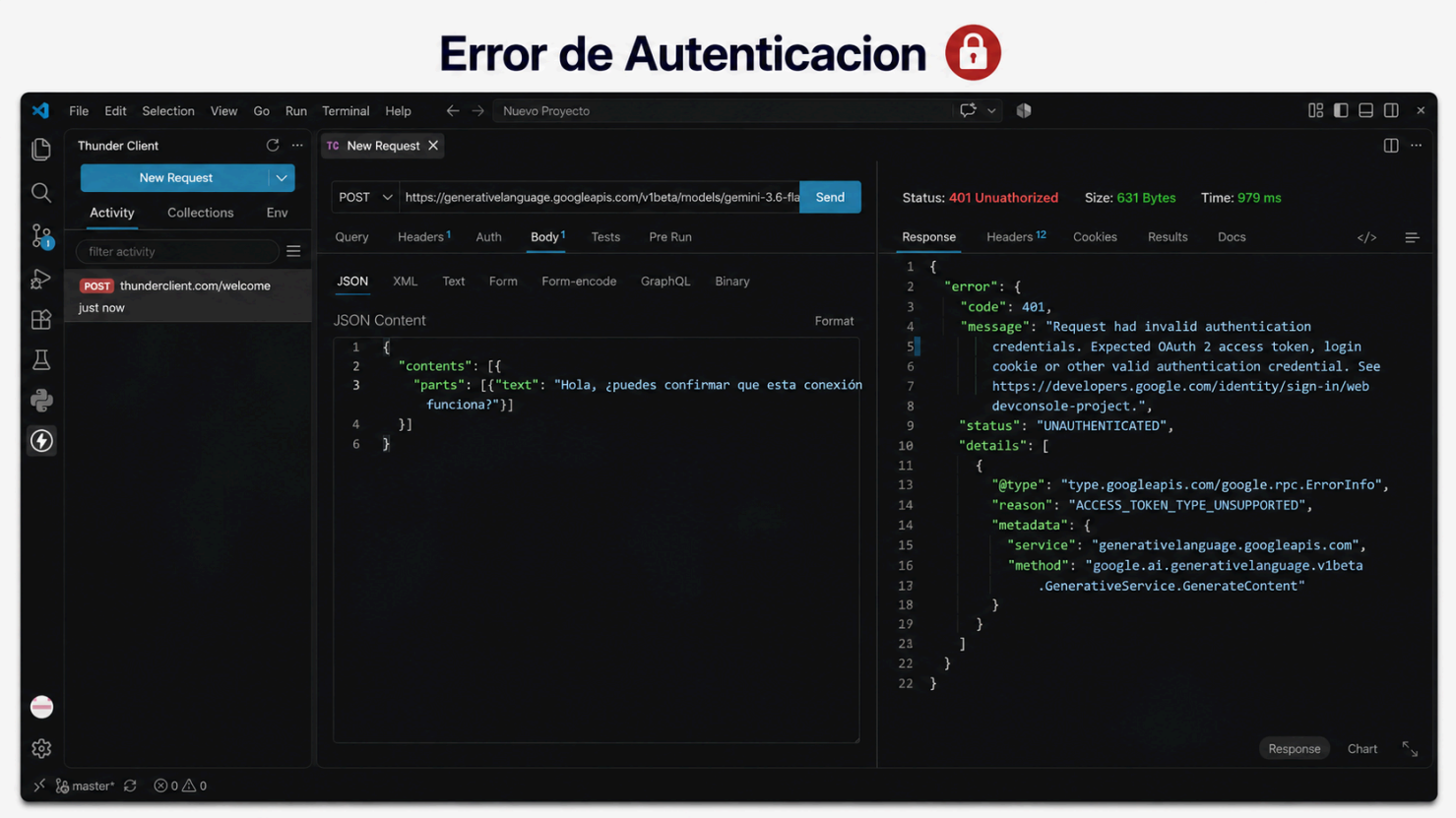
a la que muestra la documentación que estás siguiendo.

corto de tiempo. No es un error de

configuración: solo hay que esperar antes de volver a intentarlo.

Vale la pena intentar, al menos una vez, provocar un error a propósito (por ejemplo, borrando un carácter de tu credencial antes de enviar la petición) solo para ver con tus propios ojos cómo se ve ese error específico.

> 🎯 Reconocer estos patrones ahora, en un entorno controlado y sin nada en juego, es exactamente la misma habilidad que vas a necesitar en la Unidad 1.3, cuando tengas que programar el manejo de estos mismos errores dentro de tu propio código.



🔎 Leer la respuesta: qué significa cada parte

Cuando envías la petición, la respuesta trae, igual que la petición, sus propias partes que conviene saber leer. El código de estado es lo primero que hay que mirar:





200 · Éxito

Un código en el **rango de los 200** indica que la petición se procesó correctamente.



400 · Error en la petición

Un código en el **rango de los 400** indica un problema con la petición que enviaste. Una credencial inválida o mal formada suele caer aquí.



500 · Error del servidor

Un código en el **rango de los 500** indica un problema del lado del servidor del proveedor, no tuyo.

El cuerpo de la respuesta, normalmente en formato **JSON**, trae el resultado real cuando la petición fue exitosa, o un mensaje de error específico cuando no lo fue: ese mensaje suele ser más útil que el código de estado solo, porque describe la causa exacta del problema.

Para la petición de ejemplo de más arriba, una respuesta exitosa (200) se ve, en términos concretos, así:

```
{
  "id": "chatcmpl-9F8x2K3mZpQ7vLxYtRn1abCdE",
  "object": "chat.completion",
  "created": 1737400000,
  "model": "nombre-del-modelo",
  "choices": [
    {
    "index": 0,
    "message": {
        "role": "assistant",
        "content": "¡Sí! La conexión funciona correctamente."
    },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 14,
    "completion_tokens": 9,
    "total_tokens": 23
  }
}
```

💡 El formato JSON en el que llega esta respuesta merece una mención aparte, porque lo vas a ver constantemente durante todo el programa: es una forma de estructurar información en pares de

nombre y valor, anidados unos dentro de otros cuando hace falta, que tanto las personas como los programas pueden leer con facilidad.

Por ahora basta con que sepas identificar dónde está, dentro de esa estructura, el resultado generado que te interesa (normalmente bajo un nombre como el texto de la respuesta, o el contenido del mensaje), sin necesidad todavía de procesarlo con código: eso es exactamente lo que vas a aprender a hacer en la siguiente unidad.

Esta forma de leer una respuesta (código de estado primero, luego el cuerpo) no es exclusiva de esta unidad: es la misma lógica que vas a aplicar en la Unidad 1.3, cuando tu propio código en Python tenga que procesar la respuesta de un servicio externo y decidir qué hacer según lo que reciba.

✅ Checklist de verificación rápida antes de enviar

✓ **URL del endpoint**: Confirmar que coincide exactamente con la documentada por tu proveedor.

✓ **Método HTTP**: Verificar que el selector esté configurado en POST.

✓ **Header Content-Type**: Comprobar que esté definido como application/json.

✓ **Credencial de autenticación**: Asegurarse de haber pegado la llave real desde el archivo .env en el header correspondiente.

✓ **Antes de presionar "Send", confirma**: la URL del endpoint es la correcta según la documentación de tu proveedor; el método está en la opción que envía datos (no en la de solo consultar); el encabezado "Content-Type" está en "application/json"; y el encabezado de tu credencial (según la tabla de arriba) tiene tu llave real, copiada desde tu ".env".

🧭 **Siguiente actividad**

Ejecución de tu primera petición · Actividad core

Con esta base ya clara (cómo se arma la petición, dónde va la credencial, cómo se leen tanto los errores más comunes como una respuesta exitosa), la siguiente lección de este mismo **bloque** es donde ejecutas tú mismo esta prueba, con tu propia credencial, para cerrar formalmente la **Unidad U1.2**.

Última modificación: jueves, 24 de septiembre de 2026, 12:01